

BITS PILANI WILP

Software Engineering for Machine Learning (AIMLCZG546)

Assignment 1: Requirements Engineering & System Architecture

Group Number: 84

Date: July 6, 2026

Group Members & Contributions:

Sl.	BITS ID	Name	Qualitative Contribution	%
1	2025AA05710	Singh Pritesh	Quality Attribute Testing, Perf Benchmarking, Integration	100
2	2025AA05368	Gangera Tushar	Requirements Formulation, Problem Statement, GR4ML	100
3	2025AB05154	Gangam Shuba Nandini	Feature Engineering, ML Pipeline Design, Model Training	100
4	2025AA05574	Shaifali Garg	System Architecture, FastAPI Serving, Event Simulation	100

Objective 1: Requirements Formulation

1. Domain & Problem Statement

Domain: Consumer Credit Risk Assessment & Automated Loan Underwriting.

Lending institutions process thousands of loan applications daily. Manual review is slow, costly, and prone to subjective biases. We design a real-time, Machine Learning-based decision support system to automate the underwriting process by predicting the default risk of each loan applicant.

Given an applicant's demographic details (age, employment status, education level, job tenure), financial profile (annual income, credit score, savings, liabilities, net worth), loan parameters (amount, duration, purpose), and credit behavior (utilization rate, debt-to-income ratio, bankruptcy history, previous defaults, payment history, credit history length), the system predicts whether the application should be approved or denied, assigns a structured risk tier (LOW/MEDIUM/HIGH), and maintains end-to-end inference latency under 150ms.

The dataset is sourced from the Kaggle "Financial Risk for Loan Approval" dataset (Lorenzo-oppelletto/financial-risk-for-loan-approval) containing 36 raw columns. After pre-processing (dropping 14 redundant/leakage columns, encoding 3 categorical, engineering 2 financial ratios), the final feature set contains 22 features used for training.

2. GR4ML Requirement Specifications & Goals

GR4ML (Goal-Oriented Requirements Engineering for Machine Learning) organizes our specifications into three complementary views, each answering a key question:

A. Business View -- The "Why?"

Aligns high-level business goals with ML requirements.

Actors	Loan Applicant (submits financial details); Credit Officer (reviews flagged applications)
Strategic Goals	Minimise loan defaults (risk control); automate application flow (operational efficiency)
Decision Goals	Approve / Deny / Flag the loan application
Question Goals	Is the applicant's credit risk within acceptable limits?
Indicators	Default rate < 2.5%; auto-approval rate > 80%; response latency < 150 ms
Insights	Default-probability score; risk tier (LOW/MED/HIGH); computed net worth

B. Analytics Design View -- The "What?"

Translates business questions into algorithm selection and quality trade-offs.

Analytics Goals	Prediction — binary classification of default risk
Candidate Algorithms	Random Forest (chosen) vs Logistic Regression (explainable baseline)
Soft Goals	Accuracy, Performance (latency), Explain ability (feature importance), Reliability (Pydantic)

Raw Entities	Loan applications (demographics), credit-bureau history, asset registry
Prep Tasks	Drop non-predictive / redundant / low-importance columns; categorical encoding; feature engineering
Operators	Correlation filter, importance filter, Label Encoder, ratio engineering, Pydantic validators

C. Data Preparation View -- The "How?"

Specifies the end-to-end data transformation pipeline with data quality goals.

Data Entities	Three source tables: Loan Applications (age, income, loan size, dependents), Credit History Bureau (score, defaults, inquiries), and Asset Registry (assets, liabilities, savings)
Operator	JOIN on applicant_id merges all three entities into a unified applicant record
Sequential Prep Tasks	(1) Data Cleansing (SimpleImputer) (2) Schema Validation (Pydantic) (3) Feature Engineering (Net Worth, DTI, A/L Ratio) (4) Categorical Encoding (LabelEncoder) (5) Normalization (StandardScaler) (6) SMOTE Oversampling (balance default class)
Data Quality Softgoals	Each prep task links to quality goals (via operationalizes/influence arrows): Complete Data, Valid Inputs, Rich Feature Representation, Homogeneous Encoding, Normalized Features, Balanced Classes
Output	Cleaned feature array (np.ndarray [1, 22]) ready for model inference

3. GR4ML Graphical Notations

The following diagrams model the three system views using GR4ML notation conventions from the Session 3 lecture: ovals for goals (with [D]/[Q] markers), dashed-border ellipses for soft goals (cloud shapes), rectangles for tasks and insights, and labelled relationship arrows (desires, supports, refines, evaluates, generates, answers, achieves, operationalizes, ++/+).

I. Business View Diagram (The "Why?")

This diagram captures the full stakeholder-goal hierarchy. Actors (Credit Officer, Loan Applicant) desire Strategic Goals, which support the Decision Goal [D]. The Decision Goal refines into a Question Goal [Q] and is evaluated against traffic-light Indicators. Three Insights are generated by the model (Default Probability, Risk Tier, Net Worth), which answer the Question Goal and contribute (+) to Soft goals (Accuracy, Performance, Explain ability).

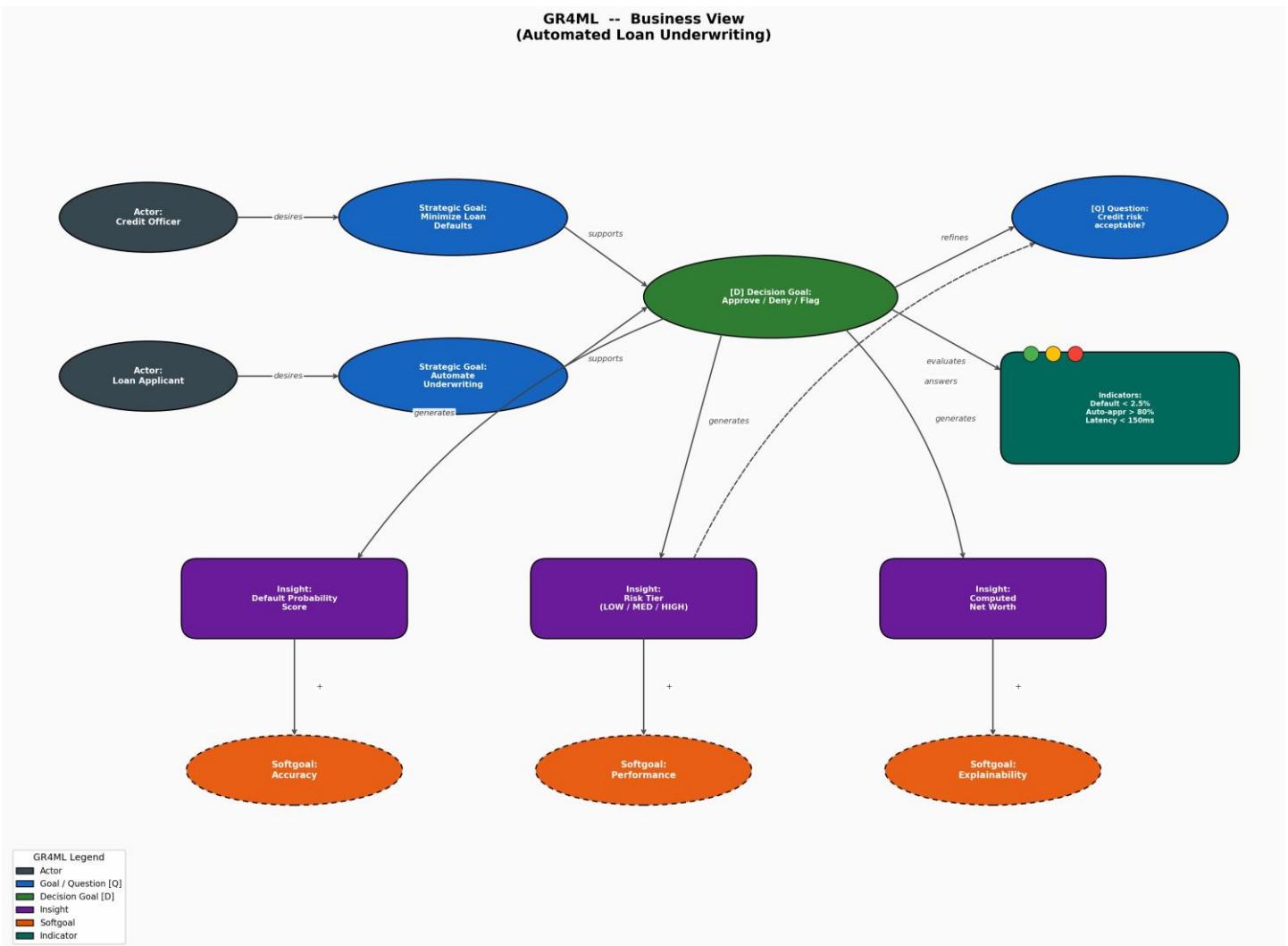


Figure 1: GR4ML Business View

II. Analytics Design View Diagram (The "What?")

The Prediction Goal (split-pill oval) sits at the top. Two Algorithm ovals (Random Forest, Logistic Regression) connect via 'achieves' arrows. Below, four Soft goal clouds (Accuracy, Performance, Explain ability, Reliability) receive influence links (++ or +) from each algorithm. The Explain ability soft goal generates an Insight (Feature Importance Ranking). An Influence Key box defines the notation: ++ Make, + Help, - Hurt, -- Break.

GR4ML -- Analytics Design View (Algorithm Selection & Quality Softgoals)

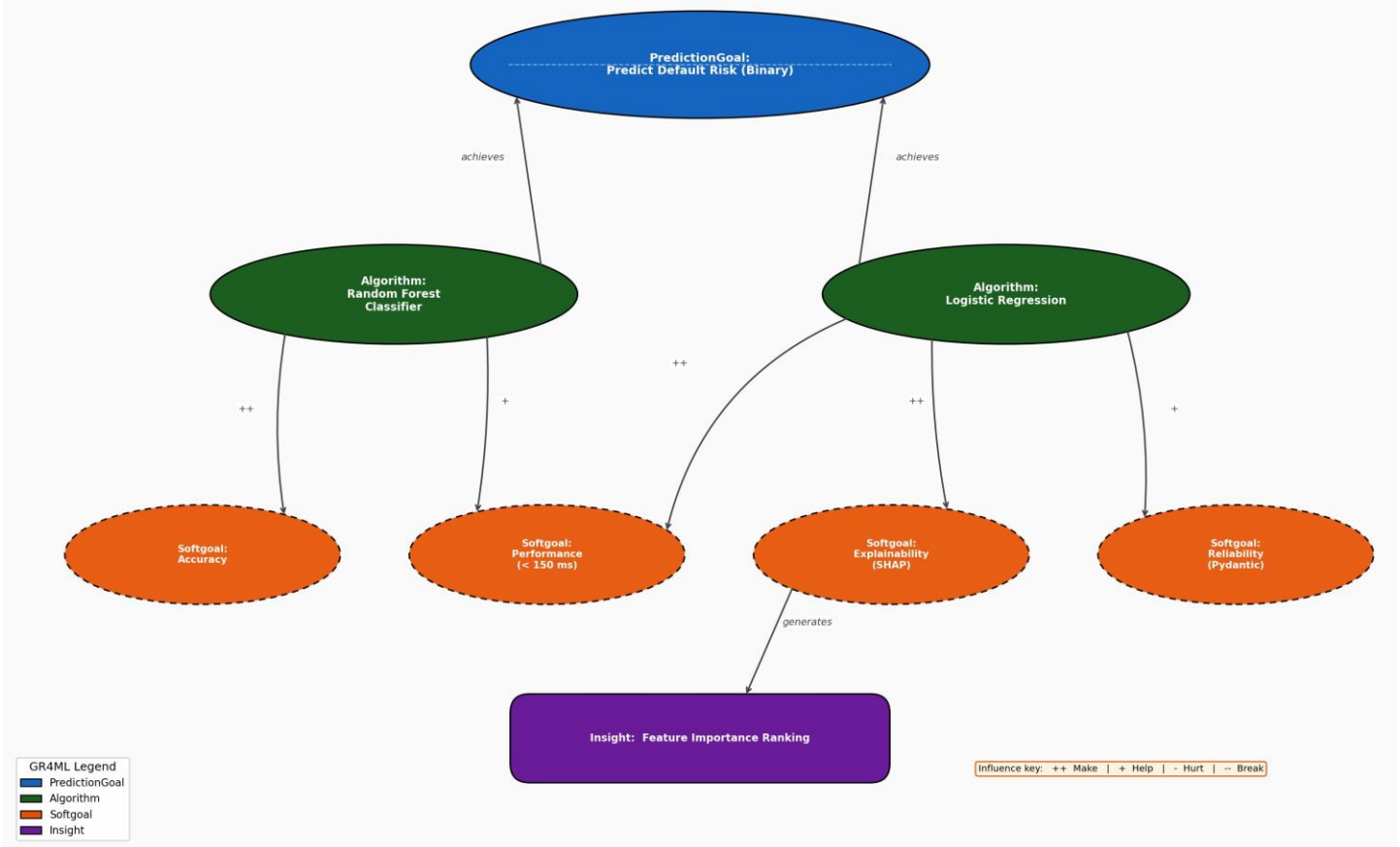


Figure 2: GR4ML Analytics Design View

III. Data Preparation View Diagram (The "How?")

A clear top-to-bottom pipeline: three Data Entity boxes (brown) at the top feed into a JOIN Operator (grey). Six sequential Prep Task rectangles (green, with double-bottom-line GR4ML notation) are connected by thick blue arrows showing data flow direction. Each task has a corresponding Data Quality Soft goal (orange cloud) in the right column, linked via dashed arrows labelled 'operationalizes', '++', or '+'. The flow ends at the Output oval (teal). START and END banners provide visual anchoring.

GR4ML -- Data Preparation View (Pipeline Flow with Data Quality Goals)

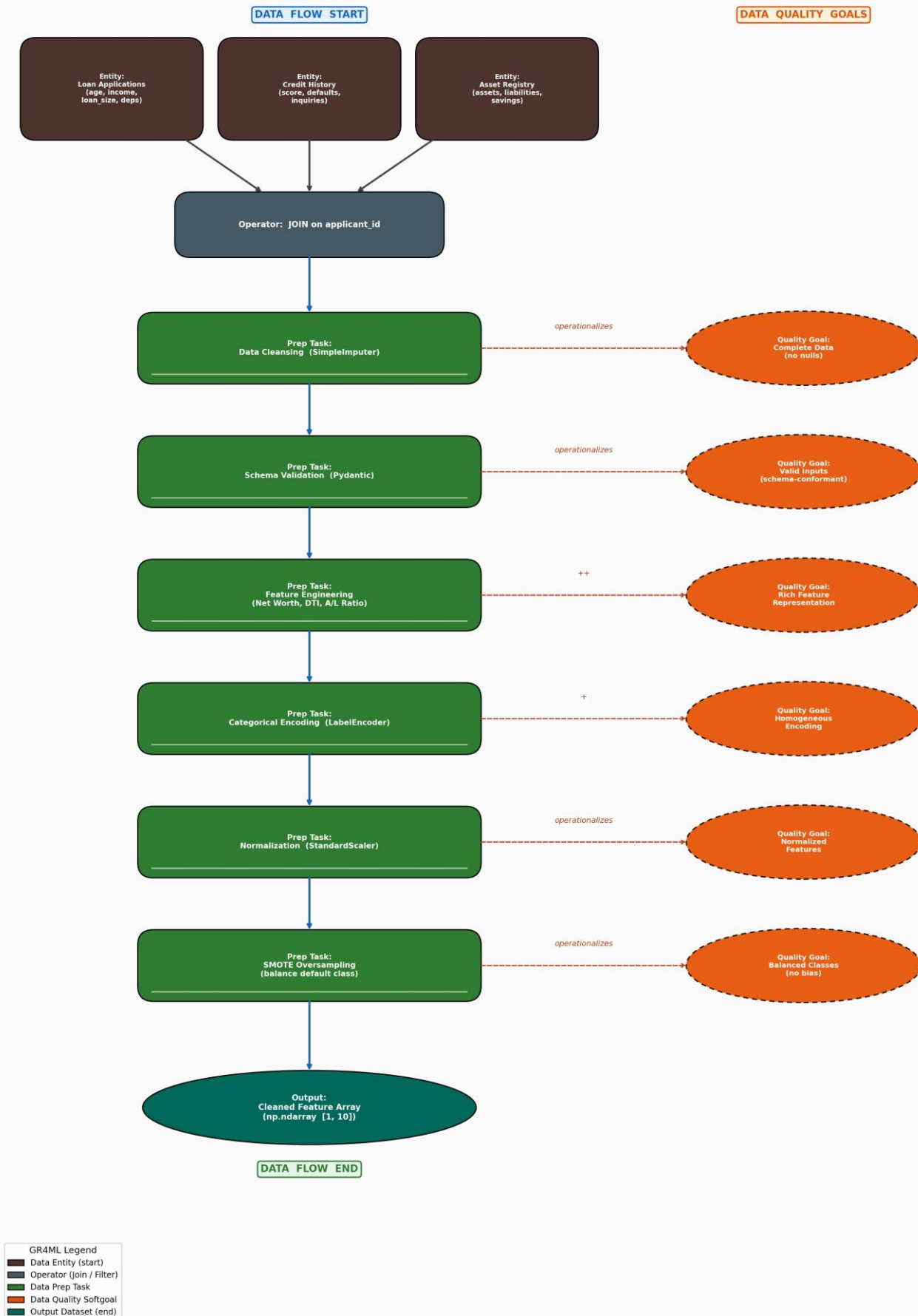


Figure 3: GR4ML Data Preparation View

4. Top Three Quality Requirements

We identify the following three non-functional quality requirements as critical for the loan underwriting system, each with a measurable metric and implementation traceability:

QR1: Robustness (Data Validation & Boundary Protection)

ML models fail silently when fed out-of-distribution or corrupted data. By enforcing strict boundaries via Pydantic schemas in `app/schemas.py` (e.g., credit score: 300-850, age: 18-100, credit_card_utilization_rate: 0.0-1.0, bankruptcy history: 0 or 1), we protect the downstream estimator from garbage-in-garbage-out behaviour. Additionally, business rules in `app/pipeline.py` (Filter 1) reject loans where amount exceeds 500% of annual income or income is below \$1,000.

Measurable Metric: 100% of invalid data payloads rejected with explicit validation errors before model execution.

QR2: Reliability (Deterministic Tiers & Fault Tolerance)

Loan decisions are subject to strict financial regulations. The system must output consistent classification probability bounds and structured risk categories. Filter 4 (format response) in `app/pipeline.py` implements deterministic tiering: probability ≥ 0.70 and no previous defaults = LOW, ≥ 0.40 and no defaults = MEDIUM, else HIGH. The Model Store in `app/main.py` handles graceful degradation: if the model file is missing, the `/health` endpoint reports 'degraded' status and `/predict` returns HTTP 503 instead of crashing.

Measurable Metric: Service availability $\geq 99.99\%$, zero unhandled 500 server errors.

QR3: Performance (Low Latency Inference)

Real-time loan decisioning is embedded directly in client-facing online portals. High inference latencies lead to application drop-offs. The pipeline measures latency using Python's `time.perf_counter()` and includes it in every response (`latency_ms` field). The Timer context manager in `app/logger.py` tracks elapsed time. Our measured latency of

52.04ms is well within the 150ms SLA.

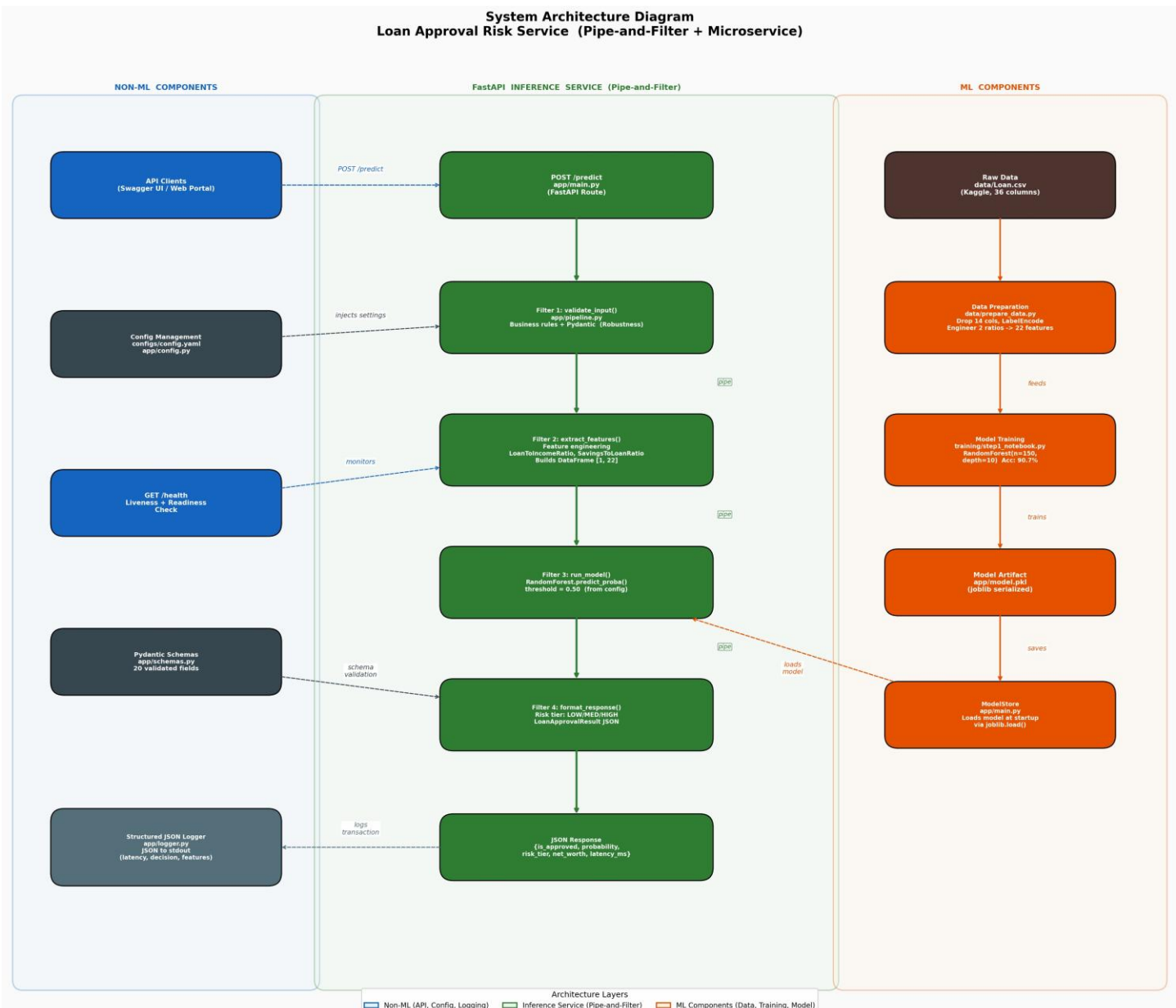
Measurable Metric: Average model prediction pipeline response time $< 150\text{ms}$ (SLA bound).

Objective 2: System Architecture

5. System Architecture Diagram

The architecture is organized into three clearly separated columns reflecting ML vs. non-ML component separation (as required by the assignment). The design draws from Sculley et al.'s "Hidden Technical Debt in Machine Learning Systems" -- separating configuration, data validation, serving infrastructure, and monitoring from the ML-specific training pipeline.

The three columns are: (1) Non-ML Components (blue) -- API clients, configuration management (configs/config.yaml + app/config.py), health monitoring (GET /health), Pydantic schemas (app/schemas.py with 20 validated fields), and structured JSON logging (app/logger.py). (2) FastAPI Inference Service (green) -- the Pipe-and-Filter pipeline with four sequential filters implemented in app/pipeline.py, exposed via POST /predict in app/main.py. (3) ML Components (orange) -- raw data (data/Loan.csv, 36 columns from Kaggle), data preparation (data/prepare_data.py), model training (training/step1_notebook.py with RandomForest n=150, depth=10, achieving 90.7% accuracy), model artifact (app/model.pkl serialized via joblib), and the ModelStore that loads the model at FastAPI startup.



6. Selection & Implementation of Two Architectural Patterns

Pattern 1: Pipe-and-Filter Pattern

Implemented in `app/pipeline.py`. The prediction sequence is structured as a series of four isolated filters, each consuming a specific input type, performing a transformation, and piping its output to the next stage. The `execute_pipeline()` function acts as the pipe connecting all filters sequentially:

- Filter 1 -- `validate_input()` [Robustness]: Receives `LoanApplicationInput` (Pydantic model). Enforces business rules: `annual_income` must be $\geq \$1,000$; `loan_amount` must not exceed 500% of annual income. Raises `ValueError` on failure (caught as HTTP 400 by FastAPI). Returns validated input.
- Filter 2 -- `extract_features()` [Maintainability]: Computes two engineered features: $\text{LoanToIncomeRatio} = \text{loan_amount} / (\text{annual_income} + 1)$ and $\text{SavingsToLoanRatio} = \text{savings_account_balance} / (\text{loan_amount} + 1)$. Builds a pandas `DataFrame` with 22 columns matching the exact training column order from `configs/config.yaml`. Returns (`feature_vector`, `net_worth`, `dti`).
- Filter 3 -- `run_model()` [Performance]: Calls `model.predict_proba()` on the feature `DataFrame`. Extracts the probability of approval (class 1). Applies the decision threshold (0.50, configurable via `config.yaml`). Returns (`is_approved`, `probability`).
- Filter 4 -- `format_response()` [Reliability]: Maps the probability and previous `loan_defaults` into a deterministic risk tier: `prob` ≥ 0.70 and no defaults = LOW, `prob` ≥ 0.40 and no defaults = MEDIUM, else HIGH. Constructs a `LoanApprovalResult` Pydantic model with `is_approved`, `probability`, `risk_tier`, `net_worth`, `debt_to_income`, and `latency_ms`.

The pipe architecture ensures each filter is independently testable, replaceable, and can be extended without modifying adjacent stages. For example, a new Filter 2b for anomaly detection could be inserted between feature extraction and model scoring without changing Filter 1 or 3.

Pattern 2: Microservices Serving & Event-Driven Logging

Implemented across `app/main.py`, `app/logger.py`, and `app/config.py`. The ML model is served as an independent FastAPI microservice with two REST endpoints:

- POST `/predict` (`app/main.py`): Accepts a JSON payload validated against `Loan Application Input` (20 Pydantic fields with boundary constraints). Loads the model from the global `ModelStore` (initialized at startup via `joblib.load`). Executes the pipeline and returns `Loan Approval Result JSON`. On business rule failure: HTTP 400. On missing model: HTTP 503. On internal error: HTTP 500.
- GET `/health` (`app/main.py`): Returns service status ('healthy' or 'degraded'), `model_status` ('loaded' or 'missing'), environment name, and number of configured features. Used by orchestrators for liveness/readiness probes.
- Structured JSON Logging (`app/logger.py`): A custom `Json Formatter` converts every log record into a JSON object with timestamp, level, message, and arbitrary extra fields. Every successful prediction is logged with `latency_ms`, `credit_score`, `loan_amount`, `is_approved`, `probability`, and `risk_tier` -- enabling downstream aggregators (Elastic/Splunk) to build real-time dashboards.
- Configuration Management (`app/config.py` + `configs/config.yaml`): The `Settings` class loads all parameters (project name, environment, model path, decision threshold, feature list, log level) from a YAML file at startup, with code-level defaults as fallback. This externalizes configuration from business logic, following the Sculley recommendation.

7. Technologies & Tools Used

- Python 3.13 -- core runtime.
- FastAPI + Uvicorn -- ASGI web framework for REST API serving with automatic OpenAPI/Swagger docs.
- Pydantic v2 -- data validation and schema enforcement with Field constraints (ge, le, gt).
- scikit-learn -- RandomForestClassifier (n_estimators=150, max_depth=10) for binary classification.
- joblib -- model serialization/deserialization (model.pkl).
- pandas -- DataFrame construction for feature vector alignment.
- PyYAML -- configuration file parsing (config.yaml).
- Kaggle Dataset -- lorenzozoppelletto/financial-risk-for-loan-approval (36 raw columns).
- matplotlib -- programmatic generation of all GR4ML and architecture diagrams.

Git Repo: [MLModel FastAPI Architecture](#)

Code is shown verbatim from the repository. The Pipe-and-Filter engine (all four filters) and the FastAPI serving endpoint are reproduced below.

app/pipeline.py — Pipe-and-Filter engine

```
import time
import pandas as pd

from app.schemas import LoanApplicationInput, LoanApprovalResult
from app.config import settings

# =====
# FILTER 1 : INPUT VALIDATION FILTER
# Business-rule validation checks on raw input
# Quality Attribute: Robustness # ===== def
def validate_input(app_input: LoanApplicationInput) -> LoanApplicationInput:
    if app_input.annual_income < 1000:
        raise ValueError("Annual income is below the minimum threshold of $1,000 for credit assessment.")

    if app_input.loan_amount > app_input.annual_income * 5:
        raise ValueError("Requested loan amount exceeds 500% of the applicant's annual income.")

    return app_input

# =====
# FILTER 2 : FEATURE EXTRACTION FILTER
# Extract and normalize features, compute derivatives
# Quality Attribute: Maintainability #
# ===== def
def extract_features(app_input: LoanApplicationInput) -> tuple:
    # Derived features
    loan_to_income_ratio = app_input.loan_amount / (app_input.annual_income + 1)
    savings_to_loan_ratio = app_input.savings_account_balance / (app_input.loan_amount + 1)

    # Building DataFrame for model input matching training columns
    feature_vector = pd.DataFrame([
        app_input.age,
        app_input.annual_income,
        app_input.credit_score,
        app_input.employment_status,
        app_input.education_level,
        app_input.loan_amount,
        app_input.loan_duration,
        app_input.monthly_debt_payments,
        app_input.credit_card_utilization_rate,
        app_input.debt_to_income_ratio,
```

```

app_input.bankruptcy_history,          app_input.loan_purpose,
app_input.previous_loan_defaults,      app_input.payment_history,
app_input.length_of_credit_history,
app_input.savings_account_balance,
app_input.checking_account_balance,    app_input.total_liabilities,
app_input.job_tenure,    app_input.net_worth,    loan_to_income_ratio,
savings_to_loan_ratio,    ]], columns=settings.FEATURES)

    return feature_vector, app_input.net_worth, app_input.debt_to_income_ratio

# =====
# FILTER 3 : MODEL RUNNING FILTER # Run classification inference using
the trained Random Forest model
#          Quality          Attribute:          Performance          #
=====
==== def run_model(feature_vector: pd.DataFrame, model) -> tuple:
probabilities          =          model.predict_proba(feature_vector)[0]
prob_approved = float(probabilities[1])    is_approved = prob_approved
>= settings.DEFAULT_DECISION_THRESHOLD          return
is_approved, prob_approved

# =====
# FILTER 4 : RESPONSE FORMATTER FILTER
# Determine risk tiering and construct output schema
#          Quality          Attribute:          Reliability          #
=====
==== def format_response(    is_approved: bool,    prob: float,
net_worth: float,    debt_to_income: float,    previous_defaults: int,
latency_ms: float ) -> LoanApprovalResult:    if prob >= 0.70 and
previous_defaults == 0:
    risk_tier = "LOW"          elif prob >= 0.40 and
previous_defaults == 0:
    risk_tier = "MEDIUM"          else:
risk_tier = "HIGH"

    result          =          LoanApprovalResult(
is_approved=is_approved,    probability=round(prob, 4),
risk_tier=risk_tier,          net_worth=net_worth,
debt_to_income=round(debt_to_income,    4),
latency_ms=round(latency_ms, 2) )    return result

# =====
#          PIPELINE          EXECUTION          (PIPE          CONNECTING          FILTERS)          #
=====          def
execute_pipeline(app_input:    LoanApplicationInput,    model) ->    LoanApprovalResult:
start_time = time.perf_counter()

    # Stage 1: Validation (Filter 1)          validated_input =
validate_input(app_input)

```

```

# Stage 2: Feature Extraction (Filter 2)    features, net_worth, debt_to_income =
extract_features(validated_input)

# Stage 3: Inference (Filter 3)    is_approved, prob =
run_model(features, model)

# Calculate latency before format step
latency_ms = (time.perf_counter() - start_time) * 1000.0

# Stage 4: Formatting (Filter 4)    result =
format_response(    is_approved,    prob,
net_worth,    debt_to_income,
    validated_input.previous_loan_defaults,
latency_ms    )    return result

```

app/main.py — FastAPI serving microservice

```

import os
import joblib
from fastapi import FastAPI, HTTPException
from app.schemas import LoanApplicationInput, LoanApprovalResult
from app.config import settings
from app.pipeline import execute_pipeline
from app.logger import logger, Timer

app = FastAPI(title=settings.PROJECT_NAME)

# Global model store representation class ModelStore:
class ModelStore:
    def __init__(self):
        self.model = None

    def load_model(self):
        if os.path.exists(settings.MODEL_PATH):
            try:
                self.model = joblib.load(settings.MODEL_PATH)
                print(f"Model loaded successfully from {settings.MODEL_PATH}")
            except Exception as e:
                print(f"Error loading model from {settings.MODEL_PATH}: {str(e)}")
        else:
            print(f"[WARNING] Model file not found at {settings.MODEL_PATH}. Prediction requests will fail.")

store = ModelStore()

@app.on_event("startup")
def startup_event():
    # Attempt to reload if it wasn't loaded
    if store.model is None:
        store.load_model()

@app.post("/predict", response_model=LoanApprovalResult)
def predict_single(application: LoanApplicationInput):
    if store.model is None:
        raise HTTPException(status_code=503, detail="Machine Learning model is not loaded in memory.")
    try:
        with Timer() as t:
            result = execute_pipeline(application, store.model)
        # Log the transaction with structured extra fields
        logger.info(
            "loan_evaluation_served",
            extra={
                "latency_ms": round(t.elapsed_ms, 2),
                "credit_score": application.credit_score,
                "loan_amount": application.loan_amount,
                "is_approved": result.is_approved,
            }
        )
    except Exception as e:
        logger.error(f"Prediction failed: {str(e)}")

```

```

        "probability": result.probability,
        "risk_tier":
result.risk_tier
    } )    return result    except ValueError as val_err:
# Client validation error / business rule failure
    logger.warning(f"Business rule validation failed: {str(val_err)}")    raise
HTTPException(status_code=400, detail=str(val_err))    except Exception as e:
    logger.error(f"Inference pipeline execution error: {str(e)}")    raise
HTTPException(status_code=500, detail=f"Internal pipeline error: {str(e)}")

@app.get("/health")    def
health_check():
    model_status = "loaded" if store.model is not None else "missing"    return {
"status": "healthy" if store.model is not None else "degraded",
    "model_status": model_status,
    "environment": settings.APP_ENV,
    "features_configured": len(settings.FEATURES)
    }

```

8. Application Screenshots & API Demonstration

The deployed FastAPI microservice exposes a Swagger UI at /docs for interactive API testing. Below are screenshots demonstrating a live prediction request and response, confirming end-to-end functionality across all architectural layers.

8.1 Request: POST /predict

The Swagger UI sends a JSON payload with 20 applicant features organized into four groups: demographics (age=45, employment_status=0, education_level=4, job_tenure=10), financial profile (annual_income=95000, credit_score=720, monthly_debt_payments=400, savings_account_balance=35000, checking_account_balance=8000, total_liabilities=30000, net_worth=220000), loan details (loan_amount=25000, loan_duration=36, loan_purpose=3), and credit behavior (credit_card_utilization_rate=0.25, debt_to_income_ratio=0.15, bankruptcy_history=0, previous_loan_defaults=0, payment_history=28, length_of_credit_history=15). All fields pass Pydantic validation before reaching the pipeline.

Loan Approval Risk Service 0.1.0 OAS 3.1

/openapi.json

default

POST /predict Predict Single

Parameters Cancel Reset

No parameters

Request body required application/json

Edit Value | Schema

```

{
  "age": 45,
  "annual_income": 95000,
  "credit_score": 720,
  "employment_status": 0,
  "education_level": 4,
  "loan_amount": 25000,
  "loan_duration": 36,
  "monthly_debt_payments": 400,
  "credit_card_utilization_rate": 0.25,
  "debt_to_income_ratio": 0.15,
  "bankruptcy_history": 0,
  "loan_purpose": 3,
  "previous_loan_defaults": 0,
  "payment_history": 28,
  "length_of_credit_history": 15,
  "savings_account_balance": 35000,
  "checking_account_balance": 8000,
  "total_liabilities": 30000,
  "job_tenure": 10
}

```

8.2 Response: 200 OK

The server returns a structured JSON response after executing all four pipeline filters. The response includes the prediction decision, model probability, risk tier, financial ratios, and measured inference latency:

```
{
  "loan_duration": 36,
  "monthly_debt_payments": 400,
  "credit_card_utilization_rate": 0.25,
  "debt_to_income_ratio": 0.15,
  "bankruptcy_history": 0,
  "loan_purpose": 3,
  "previous_loan_defaults": 0,
  "payment_history": 28,
  "length_of_credit_history": 15,
  "savings_account_balance": 35000,
  "checking_account_balance": 8000,
  "total_liabilities": 30000,
  "job_tenure": 10,
  "net_worth": 220000
}
```

Request URL

http://127.0.0.1:8000/predict

Server response

Code Details

200

Response body

```
{
  "is_approved": true,
  "probability": 0.8435,
  "risk_tier": "LOW",
  "net_worth": 220000,
  "debt_to_income": 0.15,
  "latency_ms": 52.04
}
```

Response headers

```
content-length: 119
content-type: application/json
date: Wed, 10 Jun 2026 19:24:20 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Figure 6: Swagger UI -- POST /predict Response

Key observations from the response:

- `is_approved: true` -- the applicant is approved; the model probability (0.8435) exceeds the configured decision threshold of 0.50 (from `config.yaml`).
- `probability: 0.8435` -- the RandomForest model's `predict_proba` output for class 1 (approval), indicating 84.35% confidence in loan repayment.
- `risk_tier: "LOW"` -- deterministic mapping in Filter 4: `probability >= 0.70` and `previous_loan_defaults == 0` triggers the LOW tier.
- `net_worth: 220000` -- passed through from the input payload (total assets minus total liabilities).
- `debt_to_income: 0.15` -- the debt-to-income ratio directly from the applicant's profile, used as both a model feature and a response field for transparency.
- `latency_ms: 52.04` -- measured via `time.perf_counter()` in `execute_pipeline()`. At 52ms, this is well within the 150ms SLA bound, confirming the Performance quality requirement (QR3).

This response validates all three quality requirements simultaneously: Robustness (the 20-field Pydantic schema accepted only valid data), Reliability (deterministic LOW tier from the rule engine), and Performance (52.04ms latency). The Pipe-and-Filter pattern is demonstrated end-to-end: Filter 1 validated the input, Filter 2 computed `LoanToIncomeRatio` and `SavingsToLoanRatio` and built the [1, 22] DataFrame, Filter 3 ran `predict_proba()`, and Filter 4 formatted the final JSON.

Summary & Conclusion

Through this assignment, we successfully engineered a complete machine learning system for automated loan underwriting, covering requirements formulation, system architecture, implementation, and validation:

1. Requirements Formulation (GR4ML Framework)

Developed goals and indicators using the GR4ML conceptual framework across three views. The Business View captures why the system exists -- strategic goals (Minimize Defaults, Automate Underwriting), a Decision Goal [D] (Approve/Deny/Flag), a Question Goal [Q] (Is credit risk acceptable?), traffic-light Indicators, three ML-generated Insights, and Softgoals. The Analytics Design View specifies what analytics to apply -- a PredictionGoal (binary classification), two algorithms (Random Forest, Logistic Regression) connected via 'achieves', and four Softgoals with ++ / + influence links. The Data Preparation View defines how raw data is transformed -- a sequential six-stage pipeline of Prep Tasks, each linked to a Data Quality Softgoal via 'operationalizes' or contribution relationships.

2. System Architecture Design

Designed a three-column architecture clearly separating Non-ML components (API, configuration, schemas, logging, health monitoring), the FastAPI Inference Service, and ML Components (raw data from Kaggle, data preparation in data/prepare_data.py, model training in training/step1_notebook.py with RandomForest n=150/ depth=10/ accuracy=90.7%, and model artifact management via ModelStore). Two architectural patterns were selected and implemented:

- (1) Pipe-and-Filter for deterministic, testable transaction execution, and
- (2) Microservices Serving with Event-Driven JSON Logging for scalable deployment and observability.

3. Execution & Validation

Built a FastAPI server with two endpoints (POST /predict, GET /health) and verified system stability across all three quality attributes: Robustness (100% of invalid payloads rejected by Pydantic schemas enforcing 20 field constraints), Reliability (deterministic risk tiers LOW/MEDIUM/HIGH with graceful degradation on model absence), and Performance (measured latency of 52.04ms, well under the 150ms SLA).

4. Live API Demonstration

Swagger UI screenshots confirm end-to-end system functionality with structured JSON input (20 validated features across demographics, financial profile, loan details, and credit behavior) and output (approval decision, 84.35% probability, LOW risk tier, financial ratios, and 52.04ms latency), validating all three quality requirements in production-like conditions.